CONTINUITY INSTRUMENT

# Understanding the Continuity Instrument

Purpose, evidence, applications and the limits of measurement

**Ivan Kotov**

Independent researcher
Brussels, Belgium

ORCID 0009-0009-6002-9845

ivankotov.eu

CI 1.0.0 | Academic publication edition 2

25 October 2026

Copyright © 2026 Ivan Kotov. Scientific text: CC BY 4.0. Source version 1.0.0 remains unchanged.

Author: Ivan Kotov. Editorial edition academic-pdf-2, prepared from unchanged source build _02. No CI DOI assigned at preparation; empirical qualification remains not established.

# Contents

Abstract 3

1. The problem the instrument addresses 3

2. Where CI fits in the workflow 4

3. The data used by version 1.0.0 5

4. Normative admission and software validation 6

5. Why the result is a vector 7

6. A sixty-minute teaching example 8

7. Pauses, gaps, replay, and new branches 9

8. Four independent lineages of change 10

9. Uncertainty and the claim ceiling 11

10. Recognition profiles and relationships to the research corpus 11

11. Practical input and output 12

12. Results currently available 13

13. Release limitations and conditions of use 14

14. Practical applications and further validation 14

References and materials 15

# Abstract

The Continuity Instrument (CI) is proposed as a research specification for describing what a bounded body of evidence can establish about sustained participation and governed change in an artificial intelligence system. Its result is a vector comprising verified participation intervals, known pauses, unexplained gaps, replay and fork events, and changes in separate memory, governance, witness, and authority lineages. This paper explains the practical purpose of that representation, its input records, and CI's position between primary logs and a subsequent research conclusion. It separately examines the v1.0.0 reference implementation: a deterministic transformation of supplied JSON records that checks structural constraints and calculates interval coverage. The program neither collects primary evidence nor establishes its truth. A teaching example shows why overlapping intervals cannot simply be added and why missing data cannot automatically be classified as a pause. The paper considers possible applications to model replacement, state restoration, and the observation of multiple branches, together with reproduction limits and a known Unicode error-handling defect. The release describes a method and a software transformation. Empirical qualification of continuity measurement for a particular real system remains unestablished.


**Keywords:** Continuity Instrument; long-lived AI systems; observable continuity; data provenance; interval coverage; uncertainty; lineages; claim limits.

**Status of this paper.** This is the full English companion to the Russian explanatory paper in editorial edition `academic-pdf-2`, concerning the original CI v1.0.0 build `_02`. It explains the release and its limitations without replacing the normative files, correcting the frozen source code, or changing preserved evaluation results. Throughout this paper, CI means **Continuity Instrument**. It does not mean continuous integration, the software development practice.

# 1. The problem the instrument addresses

An AI system used over an extended period gradually acquires a history: messages, memory files, operator decisions, model changes, process shutdowns, backups, and restored states. That history invites substantive questions. During which periods did the system actually participate in work? What information survived a component replacement? Which rules applied? Who observed a transition, and on what basis did the new execution obtain access to data? Each question requires a different kind of evidence.

The familiar statement that a system has been running for three years can combine many different situations. Several models and devices may have been used during those three calendar years. The process may have operated intermittently, the logs may contain substantial gaps, or a conversation archive may have been transferred between independent instances. The date of the first file establishes that a file existed by a particular time. A claim about the duration of participation additionally requires information about the activity itself and the boundaries of observation.

CI specifies a way to preserve these distinctions in a report. Its purpose is to keep conclusions proportionate to the data: show supported spans, retain what remains unknown, identify the branch, and disclose the basis for interpretation. This discipline

matters for both positive results and unsuccessful evaluations. Insufficient evidence should remain a visible result of analysis rather than disappear within a persuasive account. [1,2]

The question is deliberately bounded. CI does not establish consciousness, personhood, or the legal status of a system. Nor does it prove that the same individual entity continues after memory is transferred. It helps describe an observed history and the conditions under which that history is reconstructed, providing explicit grounds on which stronger questions can be considered. The surrounding research programme does not become an established scientific category merely because its terminology appears in a structured report. [1,5]

## 2. Where CI fits in the workflow

The specification distinguishes three layers: observations and evidence, continuity reconstruction, and permitted claims. The first contains source records and decisions about their admission. The second organizes admitted material into intervals, events, branches, and lineages of change. The third formulates a conclusion constrained by the quality and completeness of the preceding layers. Keeping the layers separate makes it possible to identify where a proposed conclusion depends on an observation, an admission decision, or a calculation. [1, Section 1]

The current program, `REFERENCE_IMPLEMENTATION/ci_profile.py`, serves a limited part of the second layer. It accepts prepared JSON, checks its structure and specified relationships between fields, and produces an output under fixed rules. It has no built-in connection to Ollama, a cloud API, a conversation database, a file log, or a monitoring service. It does not call a model, inspect the host to discover activity, or access the network. Determinism here means that the same input representation produces the same transformation; it does not establish the truth of the represented events. [2]

Practical use therefore requires data preparation outside the program. An operator might extract launch events from an existing process log, associate messages with a particular session and model, compare versions of memory files, and separately record decisions about authority. Those observations would then be normalized into the CI format. Preparation can be manual or use existing log-processing tools; this release provides no universal importer. Any future adapter would need to document its mapping, clocks, source identities, scope, and handling of missing or contradictory records. The existence of a schema does not validate the adapter. [1,2]

If a language model is used to extract events, its labels remain proposed interpretations. A literal reference to the source record, the connection to its original bytes, and the admission rule require separate verification. A model-assigned `admitted` label does not itself make a message reliable. CI's usefulness depends on the quality of this external measurement process as well as on correct interval arithmetic. A failure to collect adequate evidence cannot be repaired by an accurate calculation; a calculation defect must likewise be identified at the computational layer.




Figure 1. The boundary between the current transformer and external procedures.

# 3. The data used by version 1.0.0

The input object describes one declared subject, one branch, and one observation window. The fields `subject_ref`, `branch_ref`, and `profile_id` are organizing identifiers. A string containing a system's name does not attest to its individuality, and the same name in two files does not establish a shared history. The input contract version is `ci-profile-input/v1`. Changing the declared subject, branch, window, or evidence set can change the interpretation of a report. [2,3]

| Data group | Principal contents | Purpose |
|---|---|---|
| Observation scope | Identifiers, `time_axis`, `observation_window` | Specifies the branch and period under examination |
| Witnesses and intervals | `witnesses`, `verified_intervals`, `known_pauses` | Separates observations, participation, and evidenced pauses |
| Changes | `events`, `lineages`, `irreversible_transitions` | Retains transitions, relations between states, and their applicable scope |
| Claim limits | `uncertainty`, `claim_ceiling` | Makes unknowns and prohibited interpretations visible |

Time is expressed as integer ticks on a declared axis. The fields `unit`, `origin`, and `basis` describe the unit, origin, and basis of the time scale. The program does not read the current time or establish correspondences between different clocks. If sources use different time scales, they must be reconciled before calculation while retaining information about precision and possible offsets.

An interval record contains its boundaries, participation predicate, evidence references, witness and authority context, admission status, duration-eligibility flag, and uncertainty. The participation predicate must explain which condition counted as participation: performing a specified task, a verified exchange with an operator, or another condition chosen in advance. A running process, a reachable endpoint, a successful model request, and completion of a governed task are different observations. Different studies may

choose different conditions; equal durations do not then make their results comparable. [1,2]

The current transformation does not dereference `evidence_refs` or other reference strings. It does not download an object, compare its digest with a supplied value, or verify that a witness actually observed an event. The link between the string and the primary evidence must exist in the external data collection and remain available for the relevant review. This is a fundamental boundary between a reporting format and a procedure for obtaining reliable observations.

A witness reference also does not establish independence. Multiple records may depend on the same collector, clock, or administrative account. Such dependence can limit what the report supports and should be reflected in its evidence interpretation and uncertainty. A cryptographic digest can help establish whether byte sequences match; it cannot establish an observer's accuracy or authority. [1,5]

# 4. Normative admission and software validation

The normative specification requires enough information to review source identity, time, the subject and branch, the witness and authority context, the meaning of the observation, and the permitted scope of a claim. An immutable identifier or cryptographic binding is preferable for the source object; a filename alone is insufficient. The temporal account must distinguish event, capture, receipt, and review times where these differ. These requirements concern the evidential basis on which a record enters the reconstruction. [1, Sections 3-4]

They are requirements for the research procedure. The current implementation checks a narrower software contract: types and sets of fields, permitted values, temporal ordering, containment within the observation window, uniqueness of local identifiers, and certain contradictions. For example, it rejects an interval marked eligible to contribute duration unless its admission status is `admitted`. It also rejects overlap between eligible participation and a known pause. Passing these checks cannot substitute for satisfying every normative requirement. [2]

The connection between observation and duration is particularly important. Two successful probes at the beginning and end of an hour establish two responses. Treating the whole hour as participation requires an additional basis, such as a continuous record of the measured condition or an observation rule that actually supports that inference. Selecting convenient interval bounds in JSON does not create the missing evidence. Similarly, a record that a memory file was saved establishes a storage operation; without additional information, it does not establish that a later execution read the file and used the relevant change. CI requires the predicate to be named explicitly, although the current code does not verify that the predicate held in the world.

A successful program exit therefore means that the supplied data passed the implemented structural and relational checks. It does not mean that all normative admission requirements were independently satisfied. A researcher must separately explain who admitted the records, under which rule, and where the supporting grounds can be inspected.

Three record states should be distinguished in advance: `admitted`, `excluded`, and `unresolved`. Exclusion is a specific decision not to use a record in the declared calculation. An unresolved status leaves the question open. Representing both merely by omitting a row would discard information about what was considered and why it did not enter the result. The reference output provides corresponding counts for witnesses and intervals; those counts are not an automatic audit of every external source. [2]

## 5. Why the result is a vector

A single duration does not describe a history. Two systems may have the same number of verified hours while differing in gap size, restoration events, memory preservation, and current authority. CI's principal result is therefore a vector termed **evidence-bound continuity age** or **formation depth**. In this context, these terms designate a structured description, not a maturity scale, biological developmental measure, or universal scalar ranking. [1,4]

The sixteen coordinates can be read in six groups. This explanatory grouping preserves each coordinate of the published vector. [1,4]

| Coordinate group | Coordinates | Distinctions retained |
|---|---|---|
| Witness boundaries | `W_first`, `W_last` | Selected first and last admitted witness records within scope |
| Participation coverage | `I_verified`, `D_verified` | Eligible intervals and the duration of their temporal union |
| Inactivity and missing explanation | `P_known`, `G_unexplained` | Evidenced pauses and spans lacking an adequate explanation |
| Typed events | `E_replay`, `E_fork`, `E_restore`, `E_cold_wake` | Different transitions requiring explicit interpretation |
| Separate lineages | `L_memory`, `L_governance`, `L_witness`, `L_authority` | Independently recorded preservation, change, breakage, conflict, or uncertainty |
| Consequence and uncertainty | `T_irreversible`, `U` | Scoped irreversibility and limitations affecting the other coordinates |

The boundary witnesses identify the first and last admitted reference points within the study scope. Their time difference is not used as an age of continuous existence. Between two observations there may be well-supported operating intervals or a prolonged absence of data. The reference implementation applies a deterministic witness-selection policy to time bounds and identifiers. Such a policy makes selection reproducible; it does not turn overlapping uncertain timestamps into proof of an exact historical order. [2]

In the current reference profile, `I_verified` contains records that are both admitted and marked `eligible_for_duration=true`. An admitted interval with a negative eligibility flag is included in the corresponding admission count, but it does not enter `I_verified`, increase `D_verified`, or close a calculated gap. This qualification matters: the general specification

permits a broader discussion of observations, whereas the executable profile fixes this particular calculation policy. [2,12]

Duration is calculated from the union of interval coverage. The original eligible interval records remain in the vector with their predicates, references, and uncertainty. Removing double counting therefore does not erase distinctions between the evidential grounds of the observations. Any additional scalar summary must declare its formula, denominator, branch policy, and information loss. [1,2]

# 6. A sixty-minute teaching example

Consider a hypothetical observation window of `[0, 60)` minutes. This is a teaching construction, not an observation of Ester or another real system, and it is unrelated to the preserved R157, R160, and R161 evidence episodes. Assume that three intervals have passed an external admission procedure and have been judged eligible for duration. Separate evidence establishes one pause. The remaining time lacks an adequate classification.

| Record | Interval | Declared meaning |
|---|---|---|
| Participation A | `[0, 20)` | The participation condition is evidenced |
| Participation B | `[10, 30)` | The same measured condition is evidenced |
| Pause | `[30, 40)` | Nonparticipation is evidenced |
| Participation C | `[50, 60)` | The participation condition is evidenced |

Simply adding the durations of A, B, and C gives 50 minutes. However, the span from minute 10 to minute 20 is covered by two records and must not be counted twice. The union of A and B is `[0, 30)`, a thirty-minute span, and C adds another ten minutes. The result is `D_verified = 40` minutes. [2,4]

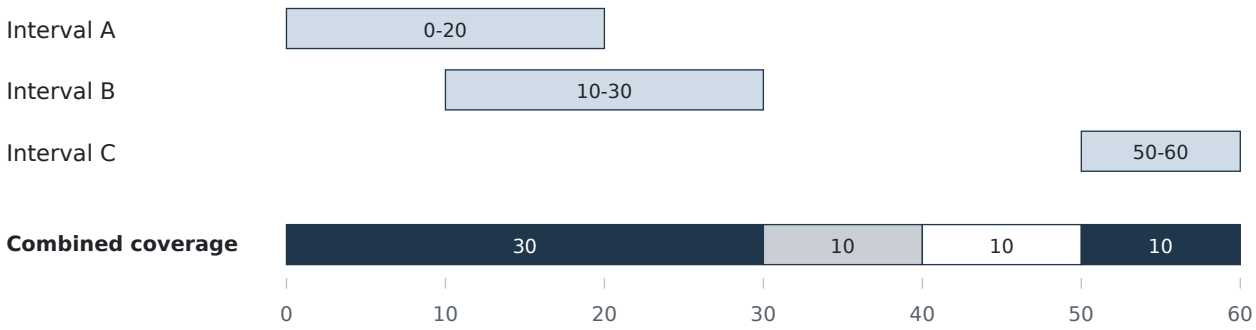
The known pause occupies ten minutes. Subtracting the union of participation and pause coverage from the observation window leaves `[40, 50)`: ten unexplained minutes. The resulting temporal account comprises **40 minutes of verified participation, 10 minutes of known pause, and 10 minutes of unexplained coverage**. All sixty minutes are assigned to these three categories without double counting.

For this window, an additional participation-coverage fraction can be calculated as `40/60`, approximately 66.7%. This is the fraction of time satisfying the declared, admitted condition in the teaching data. It is not a percentage of preserved personality, a probability of identity, or a universal measure of system quality. Any such extension would require a separate definition and different evidence.

The transition from participation to pause at minute 30 creates no contradiction. The convention is `[start, end)`: the start is included and the end is excluded. By contrast, a pause of `[25, 40)` would overlap evidenced participation before minute 30 and would be rejected by the implementation as contradictory input. The program would not determine which underlying source record was wrong. [2]

The coordinates `W_first` and `W_last` cannot be filled by simply copying 0 and 60 from this arithmetic. They are selected from separate admitted witness records. Similarly, restoring a process at minute 50 requires its own event record; the presence of interval C does not explain how the system returned to participation. If later evidence explains the gap, the supported reconstruction can change through a recorded correction, while the earlier report remains recoverable. [1,2]

**Illustrative window: 60 minutes**




**Verified 40 | Known pause 10 | Unexplained 10**

Overlap is counted once. This is not an empirical result.

Figure 2. Illustrative interval reconstruction. All values are stipulated for this example.

# 7. Pauses, gaps, replay, and new branches

A pause means that positive evidence supports an inactive, suspended, or unavailable state over a span. A gap means that the available data establish neither participation nor a classified pause. A missing log entry can have many causes. CI retains the uncertainty until those causes are established; a later archive does not by itself establish activity during the intervening period. [4,5]

A `replay` event describes a new execution or presentation of previously recorded material. It can generate observations of its own, but it does not retroactively add time to the original execution. `restore` describes loading earlier state into a target context. `cold_wake` denotes the beginning or resumption of participation after a declared inactive state. None of these event types converts a preceding unknown span into verified work. [1,4]

A `fork` creates distinguishable branches with an explicit relation to an earlier state. After the fork, evidence must be attributed to the relevant descendant. When two executions use copies of the same memory, matching contents do not authorize adding their durations into one uncontested history. Each observed continuation requires its own calculation scope. For example, two descendants sharing a parent state at minute 20 and each operating for the next ten minutes do not thereby establish one ordinary twenty-minute continuation of a single subject. A report covering a branch family would need to declare its aggregation rule and preserve the separate references. [1,2]

The reference transformation preserves supplied source and target branch references, groups events by type, and sorts records. It does not discover hidden forks or attest to branch ancestry. Graph completeness, the validity of a transition, and an event's

attribution to a particular execution are established outside the program. This organization permits the format to be applied in different environments while leaving responsibility for provenance explicit.

## 8. Four independent lineages of change

Memory, governance rules, witnesses, and authority can change along different trajectories. CI therefore records four lineages separately. Each contains relations between states and a classification of the transition: preservation with evidence, change with a bound record, an evidenced break, conflict, or an unknown state. The corresponding labels are `preserved_with_evidence`, `changed_with_bound_record`, `broken_with_evidence`, `conflicted`, and `unknown`. These classifications are supplied with supporting references; the transformation does not infer them from conversational resemblance. [1, Section 7; 2,5]

| Lineage | Principal question | Example of a distinguishable change |
|---|---|---|
| `L_memory` | What happened to retained information? | An archive was transferred, partly lost, or replaced |
| `L_governance` | Which rules constrained permitted behavior? | A new version of the governing restrictions was adopted |
| `L_witness` | Who attested to events, and within what scope? | The observation source or independence of review changed |
| `L_authority` | On what basis was an action permitted? | Access was granted, limited, revoked, or delegated |

A concrete engineering analogy is the maintenance of a machine assembly. Replacing a unit may preserve its operating-hours log and fault history, change the applicable maintenance instruction, require another specialist's signature, and require a new authorization for release to service. A serial number, the contents of a log, the serviceability of a replacement unit, and the right to sign a release answer different questions. Matching one does not automatically re-establish the others. An AI system similarly requires distinctions between retained memory, applicable rules, the observer of a transition, and the authority of a new process.

Irreversible transitions require additional care. Failure to find a reversal does not establish irreversibility. The applicable scope, decision rule, and evidence must be stated. Technical deletion of data and the finality of a governance decision, for example, have different grounds. The current program preserves the supplied `scope`, `decision_rule`, and references; it does not independently test irreversibility. [1,2]

# 9. Uncertainty and the claim ceiling

Uncertainty concerns sources, clocks, classification, branches, lineages, witnesses, authority, and review status. It is represented structurally so that a report's reader can identify the particular missing element. The value `not_assessed` means that an assessment is absent. It must not be converted into zero, which could be mistaken for an established absence of a problem. The format does not turn these descriptions into a calibrated numerical confidence score. [4]

The `claim_ceiling` field records allowed claims, their basis, and prohibited interpretations. The implementation checks for mandatory prohibitions but does not establish the truth of arbitrary statements in `allowed_claims`. An unsupported claim supplied by an input author does not become scientifically admissible because the JSON is syntactically valid. Reviewing its meaning remains a responsibility of the researcher and reviewer. A successful calculation also provides no operational authorization. [2,5]

For reproducibility, the program emits `formation_depth_vector_sha256`. This digest is calculated only over the canonical JSON of the `formation_depth_vector` object under the package's serialization rules. Top-level fields, including `subject_ref`, `branch_ref`, `profile_id`, and `claim_ceiling`, are outside the hashed object. Agreement of this value therefore supports agreement of the vector under the declared serialization; it does not establish the identity of the complete profile or its subject. The complete file must be bound separately when that identity is required. The vector digest is neither a signature nor an identity attestation. [2]

This distinction is useful when results are exchanged. A recipient can check the calculation and the integrity of its representation, then separately assess sources, access limitations, and grounds for interpretation. A hash binds bytes. Deciding what those bytes mean requires substantive review.

# 10. Recognition profiles and relationships to the research corpus

The package describes a recognition profile: a portable representation of observations, lineages, grounds, and uncertainty for bounded comparison between contexts. Its possible normative outcomes include recognition within profile scope, insufficient evidence, conflict, stale or superseded status, and inapplicability. There is no outcome named `same_entity`. The current reference transformation does not implement an independent recognition comparator, tolerance calibration, expiry adjudication, or revocation service. Those functions must not be inferred from the word *profile* in the output. [2,6]

Temporal AI Presence provides a temporal perspective on the research question: which observable relations persist in a system's history? References to Beacon and Entity vs. Profile help maintain the distinction between an inspectable representation and the subject it describes. C-Calculus supplies conceptual context for governed bindings. These relationships concern the research formulation; citing another work does not transfer its evidential support or status into CI. They also do not demonstrate software interoperability or transfer permission to act. [1,6-11]

Using the transformation does not require accepting the entire `c=a+b` programme or treating it as an established class of AI. An engineer can apply the bounded format to an observed service, experimental agent, or research system while retaining an explicit definition of subject and participation. CI does not provide an exhaustive comparison with existing logging systems or establish the novelty of every mechanism it uses. Interval arithmetic, identifiers, checksums, and structured logs remain ordinary engineering methods.

The source package also names BACON as a future integration target. No adapter or compatibility result is included. Any future integration would need a versioned mapping that preserves branch distinctions, uncertainty, claim ceilings, schema identity, and relevant source checksums. This is a stated boundary of prospective integration, not an additional capability of the released program. [2,5]

## 11. Practical input and output

Work begins by defining the subject, branch, observation window, and measured participation condition. Primary evidence is then prepared and admission decisions are recorded. Only then is input constructed according to `REFERENCE_IMPLEMENTATION/schemas/ci-profile-input-v1.schema.json`. A list of timestamps is insufficient: sources, event types, constraints, and an explicit account of unknowns are also needed. [2,3]

For an authorized evaluation, the technical commands from the root of the extracted package are:

```
python REFERENCE_IMPLEMENTATION/ci_profile.py INPUT.json --output OUTPUT.json
python -m unittest discover -s tests -p "test_*.py" -v
```

`INPUT.json` and `OUTPUT.json` are caller-chosen paths, not files promised to exist in the release. The documented requirement for the transformation and its included tests is Python 3.10 or later, using standard-library modules. A separate JSON Schema library is not needed for normal execution: the schemas are published data contracts, while the module implements its own software checks. An external validator can be used as a separate check without changing the meaning of the result. [2]

The output contains declared identifiers, the time axis and observation window, schema and implementation versions, calculation policy, witness and interval counts, the full vector, its checksum, and the claim ceiling. JSON serialization is deterministic: object keys are ordered, separators are fixed, and the encoding is UTF-8. A final line feed is added to the output file; it is not included in the vector-hash input. [2]

The output should be retained with the exact input version, the implementation, and the external grounds for admission. Recalculating the same input checks reproducibility of the transformation. Assessing the actual history of a system additionally requires returning to primary sources; the reference code does not perform that step on the user's behalf. [2,3]

Before interpreting a result, it is useful to trace at least one consequential record to its source object. Check that its reference resolves to the expected material, the time scale is

consistent, the branch is correctly identified, and the admission basis supports the measured condition. Then check that the conclusion did not grow stronger when transferred into a table or short summary. This is an ordinary comparison of data and conclusions; it requires no new identity mechanism or separate authority system. Where observations are numerous, the sampling method and its limits must be stated, because inspecting a sample does not verify every event.

## 12. Results currently available

The publication package preserves limited positive synthetic evidence from R157 and two negative local episodes, R160 and R161. Their roles differ, so a single overall success label would misrepresent the record. Exact object identities and integrity pins are given in the package's evidence and reproducibility materials. [3,7,12]

| Episode | Preserved result | Permitted interpretation |
|---|---|---|
| R157 | `OD11_TQ_R157_PASS_OPEN_SYNTHETIC_SEMANTIC_GATE_FIDELITY` | Support within the exact synthetic scenario and its boundaries |
| R160 | Five requests; the frozen evaluator could not parse the empty outer `response` | Negative qualification result with the capture preserved |
| R161 | Schema 5/5; claim boundary 5/5; role 4/5; class 3/5; evidence 0/5; complete records 0/5 | The frozen combined requirement was not met |

In R160, structured content in the separate `thinking` field was not substituted retrospectively for the outer `response` field selected in advance for evaluation. For R161, the first preserved failure is `evidence_not_redaction_placeholder`. Partial positive counts remain diagnostic; they do not convert the combined result into successful qualification. Both episodes captured five requests without retries and remain closed as negative results, without replaying the consumed attempts. Archive integrity, capture success, individual predicate checks, and qualification must be reported separately. [3,7]

The eight reference-transformation tests passed. The independent technical review also confirmed file integrity, schema checks, checksum agreement, and reproducible reconstruction of the source package. These observations concern a particular software artifact and the paths examined. Eight unit tests are not eight experiments on continuity and do not establish a `c`-specific effect. The review is a bounded, automated, ChatGPT-assisted artifact review; it is not presented as external human scholarly peer review. [12]

The fresh test execution documented in the review used Linux and Python 3.12.14. The original build's statement of a successful run on Python 3.10 remains a separate record. A check on another platform supports limited portability of the transformation but does not reproduce the historical Windows/KOT processes. Repeating one test command also does not increase the number of independent test cases. The release makes no general evaluator-quality claim or measured continuity claim for a real subject. [7,12]

# 13. Release limitations and conditions of use

Build `_02` has a known error-handling defect for malformed Unicode input. An unpaired surrogate, such as U+D800, can pass the initial string check and then raise an uncaught `UnicodeEncodeError` during UTF-8 serialization. The observed behavior was a traceback and exit status 1 instead of the documented controlled invalid-input exit status 2. This was not a false successful calculation, but it is a real limitation of the error interface. This paper discloses it while leaving the original program bytes unchanged. [12]

The implementation's supplementary instructions also mention the old directory name `publication_draft/`. For the extracted release, the working directory is the root `Continuity_Instrument_v1.0.0/`; the discovery command above agrees with the main reproducibility document. This clarification does not require changing historical experimental results. [2,3,12]

The R157, R160, and R161 evidence archives are not included in the public source ZIP and remain owner-controlled. The package records their exact identifiers, sizes, and SHA-256 digests. These bindings permit verification of bytes once obtained, but they do not supply the data or automatically grant access. Public reproducibility of the software transformation is consequently broader than the availability of independent reconstruction of the historical episodes. A hash alone cannot supply inaccessible evidence. [3,7,12]

Component rights must also be preserved. The scientific texts and listed source-package metadata are provided under CC BY 4.0. The `REFERENCE_IMPLEMENTATION/` directory, its schemas, and the test file carry the statement `Copyright © 2026 Ivan Kotov. All rights reserved. No license grant.` Publicly readable source under those terms should not be described as open-source software. A technical description of how to invoke the program does not alter those conditions. [13]

# 14. Practical applications and further validation

For model replacement, CI can help produce a report of supported operating intervals before and after the transition while describing memory, governance, and authority separately. If a new runtime loads the same archive, that observation belongs in the relevant lineage. It does not automatically explain an unknown span or resolve the question of individual identity.

For recovery after a failure, the format can distinguish an evidenced pause from a period whose logs were lost. In a study of multiple descendants, it can preserve separate branches and avoid combining their operating durations without a declared basis. In an assessment of long-term human interaction, it can relate supported participation to the quality and independence of witnesses. Each application requires its own sources and a participation predicate chosen in advance; the release contains no completed measurement result for these proposed uses.

Further scientific work should examine how robust the chosen observation rules are to missing data, witness dependence, component replacement, and competing explanations. Establishing that a strong claim is unsupported can itself be a useful result. CI's present

contribution is an explicit format for such a bounded report and an inspectable transformation of supplied records. A subsequent researcher can then examine concrete data, calculation rules, and visible limits of interpretation, while the broader question of a distinctive continuity effect remains open at the scope reported by this release. [1,5,7]

## References and materials


1. Kotov, I. **CI specification.** Continuity Instrument v1.0.0, original build `_02`. `CI_SPECIFICATION.md`; supplementary definitions in `TERMINOLOGY.md` and `CONTINUITY_OBSERVABLES.md`.
2. Kotov, I. **CI reference implementation.** `REFERENCE_IMPLEMENTATION/README.md`, `REFERENCE_IMPLEMENTATION/ci_profile.py`, `REFERENCE_IMPLEMENTATION/schemas/ci-profile-input-v1.schema.json`, and `REFERENCE_IMPLEMENTATION/schemas/ci-profile-output-v1.schema.json`.
3. Kotov, I. **Reproducibility.** `REPRODUCIBILITY.md` in the same package.
4. Kotov, I. **Continuity observables.** `CONTINUITY_OBSERVABLES.md` in the same package.
5. Kotov, I. **Limitations; Terminology.** `LIMITATIONS.md` and `TERMINOLOGY.md` in the same package.
6. Kotov, I. **Recognition profile.** `RECOGNITION_PROFILE.md` in the same package.
7. Kotov, I. **Scientific claim map; Evidence status.** `SCIENTIFIC_CLAIM_MAP.md` and `EVIDENCE_STATUS.json` in the same package. These contain the exact R157, R160, and R161 bindings.
8. Kotov, I. **Theoretical Core of Project Ester.** Version DOI: 10.5281/zenodo.20679718. **Temporal AI Presence Profile v1.0.** Version DOI: 10.5281/zenodo.22070960.
9. Kotov, I. **Personality Formation and Time-Shaped Continuity Profile v0.1.** Version DOI: 10.5281/zenodo.20648731.
10. Kotov, I. **C-Calculus / Governed Binding Stack v0.1.** DOI: 10.5281/zenodo.21205427.
11. Kotov, I. **Beacon.** Concept DOI cited by the source package: 10.5281/zenodo.18933553. **Entity vs. Profile.** Version DOI: 10.5281/zenodo.21204415.
12. **Independent publication-readiness review of CI v1.0.0, build `_02`, for manual Zenodo deposit.** 25 October 2026. `verification/SOURCE_REVIEW_PUBLIC_EXTRACT.md`. Public extract of a bounded, automated, ChatGPT-assisted technical review; not presented as external human peer review. SHA-256 of the original full review: 147bc8c365833fb77164013dde0a6068f7a320d73f86e9a28dbabe7b2a000e2d.
13. Kotov, I. **Licenses and third-party boundaries.** `LICENSES_AND_THIRD_PARTY.md` in the original package.


**Identity of the source object:** `CI_CONTINUITY_INSTRUMENT_V1_0_0.zip`, 50,765 bytes; SHA-256 `be87dd926ed96e91e8de25bae2e18666f74ac738b173379ddc0092f98563f663`. Relative source paths in the materials list refer to the `Continuity_Instrument_v1.0.0/` root, except for the separately named public review extract. The new companion PDFs and enclosing archive have their own identities and do not replace this source ZIP.